Sign in with Google

Complete setup and implementation guide — Google Cloud Console through working code

Project: **Blinto** URL shortener · NestJS backend (`:3000`) + Next.js frontend (`:3001`) · Passport `google-oauth20` with self-issued JWTs

> **Who this is for.** You, in six months, having forgotten all of it. Every value you must click, copy, or type is written out literally. Follow it top to bottom and Google sign-in will work; the final section is a checklist to prove it does.

CONTENTS

1. How the flow works

Google never gives your server a password. It gives you a *one-time code* proving someone authenticated, which your server trades — server to server, using your client secret — for that person's profile. You then issue **your own** tokens and throw Google's away. Google's role ends at "yes, this is really them."

Browser	→ clicks the button; the page navigates to `:3000/auth/google`
Backend	→ 302 to `accounts.google.com` with client_id + redirect_uri + scope
Google	→ user logs in and consents (on Google's own domain)
Google	→ 302 back to `:3000/auth/google/callback?code=…`
Backend	→ trades the code for Google's tokens, fetches the profile
Backend	→ finds or creates the matching user row in Postgres
Backend	→ signs *its own* access + refresh JWTs
Backend	→ 302 to `:3001/auth/google/callback?access_token=…&refresh_token=…`
Frontend	→ stores the tokens, cleans the URL, lands on the dashboard

> **The single most important structural point.** Steps 1 through 8 are **full page navigations**, not `fetch` calls. The user must physically see Google's consent screen, so the browser genuinely leaves your site. This is why the backend callback must end in a **redirect** and not a JSON response — no JavaScript of yours is running on that page to receive JSON. Getting this wrong is the most common way this integration fails.

2. Google Cloud Console setup

Start at console.cloud.google.com. Everything below is free and takes about ten minutes.

2.1 Create a project

1. Click the project dropdown in the top bar, then **New Project**.
2. Name it something you will recognise later — e.g. **Blinto**. No organisation needed.
3. Create it, then **make sure it is selected** in the top bar. Everything after this applies to the selected project, and configuring the wrong project is a genuinely easy mistake to make.

2.2 Configure the consent screen

Navigate to `APIs & Services → OAuth consent screen`. In the current console this is branded **Google Auth Platform**, with the settings split across *Branding*, *Audience*, *Clients*, and *Data Access*. Older guides call the same thing "OAuth consent screen" as one page — if the layout does not match, the field names still do.

1. **User type / Audience: External.** "Internal" only exists if you have a Google Workspace organisation, and restricts sign-in to that org.
2. **App name** — what the user sees on the consent screen ("Blinto wants to access your account"). **User support email** — pick your own address from the dropdown.
3. **Developer contact email** at the bottom. Required; Google emails you here about the project.
4. **Scopes:** add `.../auth/userinfo.email` and `.../auth/userinfo.profile`. These are the `email` and `profile` shorthands in the strategy code. Both are *non-sensitive*, so you will never need Google's verification review.
5. **Test users:** while the app is in *Testing*, only listed addresses can sign in — everyone else gets `access_denied`. Add your own Google account, and any teammate's, here. This traps a lot of people.

2.3 Create the OAuth client ID

Go to `APIs & Services → Credentials` (or the *Clients* tab).

1. **Create Credentials → OAuth client ID**.
2. **Application type: Web application.** Not "Desktop", not "Single-Page App" — your server holds the secret and performs the code exchange, which is the Web application flow.

3. **Authorized redirect URIs → Add URI:**

   ```
   http://localhost:3000/auth/google/callback
   ```

   This must match `GOOGLE_CALLBACK_URL` in your `.env` **character for character** — scheme, host, port, path, and no trailing slash. Google compares it as an exact string. Add one entry per environment (local, staging, production).

4. **Authorized JavaScript origins** can be left empty. That field is for browser-side flows; yours is server-side. Adding `http://localhost:3001` is harmless if you prefer.

5. Create it. A dialog shows the **Client ID** and **Client secret** — copy both now. The secret can be re-revealed later from the client's detail page, but copying now saves a trip.

> **No APIs need enabling.** You may remember having to switch on the "Google+ API" or "People API". That is obsolete — `passport-google-oauth20` reads the OpenID `userinfo` endpoint, which needs no API enabled. If sign-in fails, the cause is in section 7, not a missing API.

> **http://localhost is a special case.** Google normally rejects plain `http` redirect URIs, but permits them for `localhost` so you can develop without TLS. Every deployed environment must use `https`.

3. Environment variables

BACKEND/.ENV

```
# --- Postgres ---
PG_HOST=localhost
PG_PORT=5432
PG_USER_NAME=postgres
PG_PASSWORD=your_password
PG_DB_NAME=Blinto

# --- JWT (yours, unrelated to Google) ---
# Generate each with:  openssl rand -base64 32
JWT_ACCESS_SECRET=...
JWT_REFRESH_SECRET=...

# --- Google OAuth (from section 2.3) ---
GOOGLE_CLIENT_ID=xxxxxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxx
GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback

# Where the backend sends the browser once tokens are minted
FRONTEND_URL=http://localhost:3001
```

FRONTEND/.ENV.LOCAL

```
NEXT_PUBLIC_API_URL=http://localhost:3000
```

Variable	Must equal
`GOOGLE_CALLBACK_URL`	An Authorized redirect URI in the console, exactly
`FRONTEND_URL`	Where Next.js runs — `http://localhost:3001`
`NEXT_PUBLIC_API_URL`	Where NestJS runs — `http://localhost:3000`

Never commit this file. A client secret pushed to GitHub is compromised and must be rotated from the Console. Confirm `.env` is in `.gitignore` before your first commit, and keep a `.env.example` with the keys but no values.

The port is not arbitrary. The backend enables CORS for `http://localhost:3001` only, so the frontend must run there — `next dev -p 3001`, which is already pinned in `frontend/package.json`.

4. Packages to install

BACKEND

```
npm install @nestjs/passport passport passport-google-oauth20 \
            @nestjs/jwt @nestjs/config
npm install -D @types/passport-google-oauth20
```

The frontend needs nothing extra — it only performs a redirect and reads a query string.

5. Backend implementation

5.1 The strategy — talks to Google

Passport handles the whole handshake. Your only job is configuration plus a `validate()` that reshapes Google's profile into something your app likes. Whatever `validate()` returns becomes `req.user`.

`backend/src/auth/strategies/google.strategy.ts`

```
@Injectable()
export class GoogleStrategy extends PassportStrategy(Strategy, 'google') {
  constructor(private readonly configService: ConfigService) {
    super({
      clientID:     configService.get<string>('GOOGLE_CLIENT_ID')!,
      clientSecret: configService.get<string>('GOOGLE_CLIENT_SECRET')!,
      callbackURL:  configService.get<string>('GOOGLE_CALLBACK_URL')!,
      scope: ['email', 'profile'],
    });
  }

  async validate(accessToken: string, refreshToken: string, profile: any) {
    const { id, name, emails, photos } = profile;

    return {
      googleId:  id,
      email:     emails?.[0]?.value,
      firstName: name?.givenName,
      lastName:  name?.familyName,
      avatar:    photos?.[0]?.value,
    };
  }
}
```

The `'google'` string in `PassportStrategy(Strategy, 'google')` names this strategy so the guard can select it.

5.2 The guard — one class, two jobs

`backend/src/auth/gaurds/google-auth.guard.ts`

```
@Injectable()
export class GoogleAuthGuard extends AuthGuard('google') {}
```

Four lines, and it is worth understanding why the *same* guard is on both routes. Passport looks at whether the request carries a `?code=`:

- **No code** (the `/auth/google` hit) → build the Google URL and redirect.
- **Code present** (the callback) → exchange it, fetch the profile, call `validate()`, attach `req.user`, and only then run your handler.

5.3 Module wiring

backend/src/auth/auth.module.ts

```
@Module({
  imports: [UserModule, PassportModule, ConfigModule, JwtModule.registerAsync({ ... })],
  providers: [AuthService, AuthGuard, JwtService, GoogleStrategy],
  controllers: [AuthController],
})
export class AuthModule {}
```

Forgetting `GoogleStrategy` in `providers` is a classic. Nest never instantiates it, Passport never registers a strategy named `'google'`, and every request dies with `Unknown authentication strategy "google"`.

5.4 The two routes

backend/src/auth/auth.controller.ts

```
@Get('google')
@UseGuards(GoogleAuthGuard)
async googleLogin() {
  // Intentionally empty — the guard redirects to Google before this runs.
}

@Get('google/callback')
@UseGuards(GoogleAuthGuard)
async googleCallback(@Req() req: any, @Res() res: Response) {
  const frontendUrl = process.env.FRONTEND_URL ?? 'http://localhost:3001';

  try {
    const { access_token, refresh_token } =
      await this.authService.googleLogin(req.user);

    res.cookie('access_token',  access_token,  { httpOnly: true, sameSite: 'lax', maxAge: 15 *
    res.cookie('refresh_token', refresh_token, { httpOnly: true, sameSite: 'lax', maxAge: 7 * 2

    // The browser arrived by navigation and cannot read a JSON body,
    // so hand the tokens over in the query string.
    const params = new URLSearchParams({ access_token, refresh_token });
    return res.redirect(`${frontendUrl}/auth/google/callback?${params}`);
  } catch (error) {
    const message = error instanceof Error ? error.message : 'Google sign-in failed';
    return res.redirect(
      `${frontendUrl}/auth/google/callback?${new URLSearchParams({ error: message })}`,
    );
  }
}
```

Use `@Res()`, not `@Res({ passthrough: true })`. With passthrough, Nest still tries to send its own response after you have already redirected, and you get a headers-already-sent error.

5.5 Resolving the Google identity to a real user

Three lookups in strict order — the order is the logic:

`backend/src/auth/auth.service.ts`

```
async googleLogin(googleUser: any) {
  // 1. A returning Google user.
  let user = await this.userService.findUserByGoogleId(googleUser.googleId);

  // 2. Someone who registered with a password using the same address —
  //    attach to that account instead of creating a duplicate.
  if (!user) user = await this.userService.findUserByEmail(googleUser.email);

  // 3. Brand new person.
  if (!user) user = await this.userService.createUserByGoogleSignIn(googleUser);

  const payload = { sub: user.id, username: user.username, role: user.role };

  const access_token  = await this.jwtService.signAsync(payload, { secret: ...ACCESS,  expiresI
  const refresh_token = await this.jwtService.signAsync(payload, { secret: ...REFRESH, expiresI

  return { access_token, refresh_token };
}
```

New Google users are created with:

- `googleId` set — how step 1 finds them next time.
- `password: null` — there is no password to store. The login route checks for this and answers "*This account does not have a password. Please sign in with Google.*" instead of comparing against null.
- `username` set to their **email**. The column is unique and non-null, and Google supplies no username.
- `role: 'guest'`, same as any other new account.

> **Sign both tokens with the same claims.** The original code signed the refresh token with only `{ sub }`. Fifteen minutes later the refresh produced an access token with no `username`, and the UI crashed reading it. Two defences, both applied: sign the full payload, *and* have `refreshAccessToken` reload the user from the database rather than copying claims out of the old token. Rebuilding from the database is the stronger fix — it also keeps `role` current if it changed.

6. Frontend implementation

6.1 Starting the flow

An assignment to `window.location.href` — deliberately not `fetch`, which would only follow the redirect in the background and hand you an unreadable cross-origin response.

`frontend/src/components/google-button.tsx`

```
function start() {
  window.location.href = `${API_URL}/auth/google`;
}
```

6.2 Receiving the session

A page at `/auth/google/callback` whose only job is to read the query string, store the tokens, and get out of the way.

`frontend/src/app/auth/google/callback/page.tsx`

```
useEffect(() => {
  if (handled.current) return;   // React 18 StrictMode runs effects twice in dev
  handled.current = true;

  const failure = params.get('error');
  if (failure) { setError(decodeURIComponent(failure)); return; }

  const access  = params.get('access_token');
  const refresh = params.get('refresh_token');
  if (!access || !refresh) { setError('Google did not return a session.'); return; }

  adoptTokens(access, refresh);   // localStorage + decode for username/role

  // replace(), not push() — keeps the token-bearing URL out of history.
  router.replace('/dashboard');
}, [params, adoptTokens, router]);
```

Three details that matter more than they look:

- **The `handled` ref.** React StrictMode fires effects twice in development; without it the adoption runs twice.
- **`router.replace`.** With `push`, the URL containing both tokens stays in the back-button history and in the browser's session store.
- **Handle `?error=`.** The backend redirects failures back here too, so the user sees a message rather than a blank screen.

6.3 Why the tokens travel in the URL at all

The backend also sets both tokens as `httpOnly` cookies — but `auth.guard.ts` only ever reads `Authorization: Bearer`. Those cookies are written and never read. Since JavaScript cannot read an `httpOnly` cookie by design, the query string is the only channel left.

> **The better long-term design** is to make the guard read `request.cookies.access_token`, drop the query string entirely, and add CSRF protection (cookies are sent automatically, so a cookie-authenticated API needs it). That removes the XSS exposure of keeping tokens in `localStorage`. It is a deliberate piece of work, not a quick edit — worth doing before production.

7. Errors and what causes them

Symptom	Cause and fix
`redirect_uri_mismatch`	The most frequent one. `GOOGLE_CALLBACK_URL` differs from the console's Authorized redirect URI. Compare character by character — `http` vs `https`, a trailing slash, `127.0.0.1` vs `localhost`, wrong port. Console edits can take a minute to propagate.
`access_denied` for some users	App is in *Testing* and the address is not in Test users. Add it, or publish the app (section 9).
`Unknown authentication strategy "google"`	`GoogleStrategy` is missing from `providers` in `AuthModule`, or `PassportModule` is not imported.
Browser shows raw JSON of the tokens	The callback returns an object instead of redirecting. Use `@Res()` and `res.redirect(...)`.
`ERR_HTTP_HEADERS_SENT`	`@Res({ passthrough: true })` combined with `res.redirect`. Drop the passthrough.
Sign-in works, then breaks ~15 min later	The refresh token lacks `username`/`role`. Sign both tokens with the same payload and rebuild claims from the database on refresh.
CORS error on the API calls after landing	Frontend is not on port 3001, or the origin is missing from `enableCors` in `main.ts`. `credentials: true` forbids a wildcard origin.
Callback page loads but nothing happens	Reading `useSearchParams` outside a `<Suspense>` boundary, or the tokens are absent from the query string — check the backend's redirect URL.
`invalid_client`	Wrong client ID or secret — often the `.env` of a different Google Cloud project. Restart the backend after editing `.env`; it is read once at boot.

8. Verification checklist

Start both servers, then walk this list. Each step proves the one before it worked.

1. Backend answers a redirect to Google:

   ```
   curl -s -o /dev/null -D - http://localhost:3000/auth/google | grep -i location
   ```

   Expect `Location: https://accounts.google.com/o/oauth2/v2/auth?...` carrying your `client_id` and `redirect_uri`.

2. Open `http://localhost:3001/login` and click **Sign in with Google**. The address bar must leave for `accounts.google.com`.

3. Consent screen shows your **App name** — proof the right project is selected.
4. After consent, you land on `:3001/dashboard`, and the URL is clean (no tokens visible) thanks to `router.replace`.
5. The navbar shows your email as the username. In DevTools → Application → Local Storage, both `blinto.access_token` and `blinto.refresh_token` are present.
6. In Postgres: `SELECT id, "googleId", username, email, password FROM users;` — your row has a `googleId` and a **null** password.
7. Sign out and back in with Google: **no second row** is created (lookup 1 matched).
8. Wait 15 minutes, or clear only the access token, and use the app. It refreshes silently and the username survives — this is the bug from section 5.5 staying fixed.

9. Going to production

1. **Add the real redirect URI.** In the console, add `https://api.yourdomain.com/auth/google/callback`. Keep the localhost entry so development still works — multiple URIs are fine.
2. **Update the deployed environment:** `GOOGLE_CALLBACK_URL` to the https URL, `FRONTEND_URL` to your real frontend origin, and `NEXT_PUBLIC_API_URL` to the real API origin.
3. **Widen CORS** in `main.ts` to include the production frontend origin. With `credentials: true` you must list origins explicitly; `*` is rejected by browsers.
4. **Publish the app** (Audience → Publish). Because `email` and `profile` are non-sensitive scopes, this needs no Google verification review, and the test-user restriction disappears.
5. **Set** `NODE_ENV=production` so the cookies are flagged `secure`, and serve everything over https.
6. **Rotate every secret** that has ever been committed — client secret, both JWT secrets, the database password. Assume anything in git history is public.
7. **Turn off** `synchronize: true` in the TypeORM config and move to migrations. Left on, it will alter or drop production columns to match your entities.
8. **Consider moving auth to httpOnly cookies** (section 6.3) so tokens are no longer reachable from JavaScript.

> **The one-line summary.** Google proves *who* the person is; your backend decides *what* they can do and issues its own tokens to say so. Everything above is plumbing between those two facts.
